

5735.26 Deep Learning V26

April 26, 2026

Contents

1	Lecture 1: Foundations of Deep Learning	3
1.1	Motivation and Application Areas	3
1.2	Supervised Learning Setup	3
1.3	Model Family and Linear Regression	3
1.4	Training Objective and Loss Function	4
1.5	Gradient-Based Optimization	4
1.6	From Shallow Neural Networks to Piecewise Linear Models	4
1.7	Diagram: Accumulation of ReLU-Based Linear Pieces	5
1.8	Diagram: Input-Hidden-Output Architecture	5
2	Lecture 2: From Shallow to Deep Neural Networks	6
2.1	Supervised Learning Recap	6
2.2	Shallow Neural Network (One Hidden Layer)	6
2.3	Three Activation-Related Graphs	7
2.4	Perceptron, XOR, and Why Hidden Units Matter	7
2.5	Matrix Notation for the XOR Example	8
2.6	Vector Form for Multiple Hidden Units	8
2.7	Deep Neural Networks	8
2.8	Linear Region Capacity: Shallow vs Deep	9
3	Lecture 3: Likelihood, Loss Functions, and Probabilistic Out-puts	10
3.1	Continuation from Shallow to Deep Models	10
3.2	Dimension Check for Matrix Multiplication	10
3.3	Loss Functions and Best Parameters	10
3.4	Maximum Likelihood Principle	11
3.5	From Gaussian Likelihood to Least Squares	11
3.6	Diagram: Slope-Based Regression (Least Squares)	12
3.7	Binary Classification and Bernoulli Likelihood	12
3.8	Diagram: Probability Growth with Sigmoid	12
3.9	Multiclass Classification and Softmax	13
3.10	Cross-Entropy as Negative Log-Likelihood	13
3.11	Theoretical vs Empirical Distribution and KL	13

3.12	Dirac Delta View of Empirical Distribution	13
4	Lecture 4: Fitting Models and Optimization Algorithms	14
4.1	Fitting Models	14
4.2	Probabilistic View and Maximum Likelihood	14
4.3	Gradient Descent (GD)	15
4.4	Square Loss Example (Linear Regression)	15
4.5	Convex vs Non-Convex Loss	15
4.6	Stochastic Gradient Descent (SGD)	15
4.7	Hyperparameters	16
4.8	SGD with Momentum	16
4.9	Graph: Without vs With Momentum	17
4.10	Nesterov Accelerated Gradient (NAG)	17
4.11	Adam (Adaptive Moment Estimation)	18
4.12	Practical Notes	18

1 Lecture 1: Foundations of Deep Learning

1.1 Motivation and Application Areas

Deep learning is a family of machine learning methods used in many master-level applications:

- Computer Vision
- Natural Language Processing (NLP)
- Speech and audio processing
- Games and intelligent agents

The key idea is to learn a function from data rather than hand-crafting rules.

1.2 Supervised Learning Setup

In supervised learning, we observe a dataset

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_I, y_I)\},$$

where $\mathbf{x}_i$ is an input and y_i is the target label/value.

We aim to learn a function

$$f : \mathbf{x} \mapsto y.$$

Examples:

- Scalar input example: $x = \text{height}$, $y = \text{age category}$.
- Medical imaging example:

$$\mathbf{x} = \text{MRI image vector} = [200, 310, \dots, 0], \quad y \in \{\text{Alzheimer: yes}, \text{Alzheimer: no}\}.$$

For each training example, the model prediction is

$$\hat{y}_i = f(\mathbf{x}_i; \phi),$$

where ϕ are model parameters.

1.3 Model Family and Linear Regression

We do not search over all possible functions; we choose a hypothesis/model family. A simple family is linear regression:

$$f(x; \phi) = \phi_0 + \phi_1 x.$$

For classification, $f(\mathbf{x}_0; \phi)$ can be interpreted (after a suitable output transform) as the probability of a class, e.g., Alzheimer diagnosis.

1.4 Training Objective and Loss Function

To select the best parameters, we minimize a loss on the dataset. For squared error:

$$L(\mathcal{D}, \phi) = \sum_{i=1}^I (f(\mathbf{x}_i; \phi) - y_i)^2.$$

This is the *sum of squares* criterion.

1.5 Gradient-Based Optimization

Parameters are updated with gradient descent:

$$\phi \leftarrow \phi - \alpha \nabla_{\phi} L(\phi),$$

where $\alpha > 0$ is the learning rate.

Directional derivative intuition: for a unit direction vector $\mathbf{v}$, the rate of change is

$$\mathbf{v}^{\top} \nabla_{\phi} L.$$

If the angle θ between $\mathbf{v}$ and ∇L satisfies $\cos \theta = 1$, change is maximally increasing; if $\cos \theta = -1$, change is maximally decreasing.

1.6 From Shallow Neural Networks to Piecewise Linear Models

A shallow neural network with one hidden layer and ReLU activations:

$$h_j = a(z_j), \quad z_j = \theta_{j0} + \theta_{j1}x, \quad a(z) = \max(0, z).$$

With two hidden units:

$$f(x; \phi) = \phi_0 + \phi_1 h_1 + \phi_2 h_2 = \phi_0 + \phi_1 a(\theta_{10} + \theta_{11}x) + \phi_2 a(\theta_{20} + \theta_{21}x).$$

One possible parameter set is

$$\phi = \{\phi_0, \phi_1, \phi_2, \theta_{10}, \theta_{11}, \theta_{20}, \theta_{21}\}.$$

Each hidden unit introduces a kink at $\theta_{j0} + \theta_{j1}x = 0$. With D hidden units in 1D input, the model can form up to $D + 1$ linear regions.

1.7 Diagram: Accumulation of ReLU-Based Linear Pieces

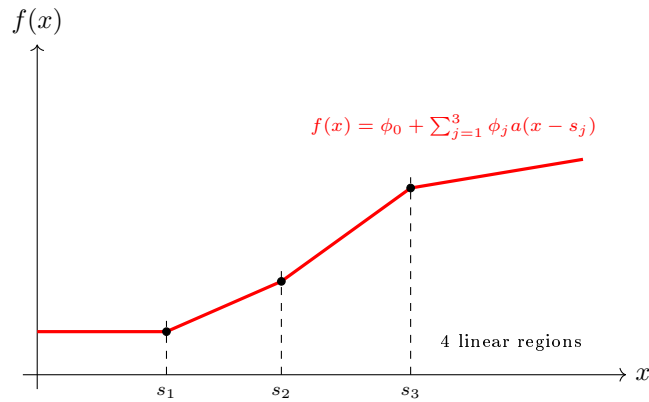


Figure 1: Combination of three shifted linear components producing three joints (kinks) in the final piecewise-linear function.

1.8 Diagram: Input-Hidden-Output Architecture

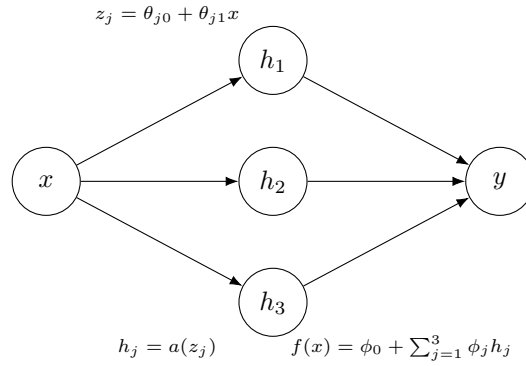


Figure 2: Shallow network with one input, three hidden ReLU units, and one output.

2 Lecture 2: From Shallow to Deep Neural Networks

2.1 Supervised Learning Recap

We mostly work with supervised learning:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I,$$

and we train a model that predicts

$$\hat{y}_i = f(\mathbf{x}_i; \phi).$$

The training objective is a loss over the dataset:

$$L(\mathcal{D}, \phi) = L(\phi).$$

2.2 Shallow Neural Network (One Hidden Layer)

For 1D input, a shallow ReLU network can be written as

$$f(x; \phi) = \phi_0 + \sum_{d=1}^D \phi_d h_d, \quad h_d = a(\theta_{d0} + \theta_{d1}x),$$

with ReLU activation

$$a(z) = \max(z, 0).$$

2.3 Three Activation-Related Graphs

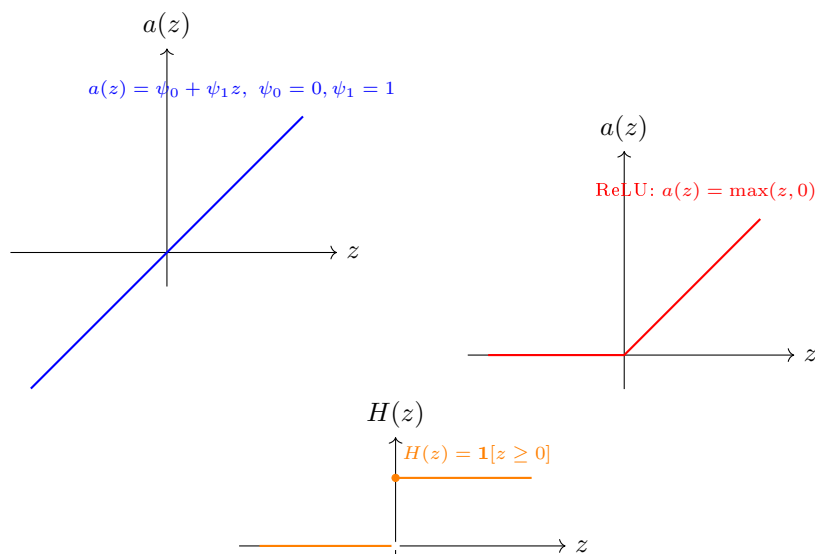


Figure 3: Left: linear activation. Middle: ReLU. Right: Heaviside step used in perceptron-style classifiers.

2.4 Perceptron, XOR, and Why Hidden Units Matter

A single linear perceptron creates one decision boundary, so XOR cannot be solved with one linear unit. With hidden units and gradient-based learning, we can represent XOR.

For the specific construction:

$$h_1 = a(0 + x_1 + x_2), \quad h_2 = a(-1 + x_1 + x_2), \quad y = 0 + h_1 - 2h_2.$$

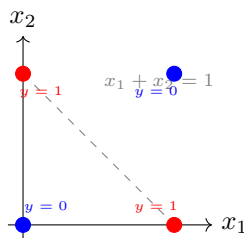


Figure 4: XOR pattern in input space: not linearly separable by one perceptron.

2.5 Matrix Notation for the XOR Example

Let

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}, \quad W_1 = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad \mathbf{b}_1 = \begin{bmatrix} 0 \\ -1 \end{bmatrix}.$$

Then hidden preactivation and hidden state are

$$\mathbf{z} = W_1 \mathbf{x} + \mathbf{b}_1 = \begin{bmatrix} x_1 + x_2 \\ x_1 + x_2 - 1 \end{bmatrix}, \quad \mathbf{h} = a(\mathbf{z}) = \begin{bmatrix} h_1 \\ h_2 \end{bmatrix}.$$

Output:

$$y = W_2 \mathbf{h} + b_2, \quad W_2 = \begin{bmatrix} 1 & -2 \end{bmatrix}, \quad b_2 = 0.$$

2.6 Vector Form for Multiple Hidden Units

For three hidden units, write

$$\mathbf{h} = \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix} = a \left(\begin{bmatrix} \theta_{10} \\ \theta_{20} \\ \theta_{30} \end{bmatrix} + \begin{bmatrix} \theta_{11} \\ \theta_{21} \\ \theta_{31} \end{bmatrix} x \right).$$

In expanded form:

$$\mathbf{h} = \begin{bmatrix} a(\theta_{10} + \theta_{11}x) \\ a(\theta_{20} + \theta_{21}x) \\ a(\theta_{30} + \theta_{31}x) \end{bmatrix}.$$

Then

$$y = \phi_0 + \begin{bmatrix} \phi_1 & \phi_2 & \phi_3 \end{bmatrix} \begin{bmatrix} h_1 \\ h_2 \\ h_3 \end{bmatrix}.$$

2.7 Deep Neural Networks

A network is called *deep* when it has more than one hidden layer.

Layer-wise notation:

$$\mathbf{h}^{(1)} = a(\boldsymbol{\beta}_0 + \Omega_0 \mathbf{x}), \quad \mathbf{h}^{(2)} = a(\boldsymbol{\beta}_1 + \Omega_1 \mathbf{h}^{(1)}), \quad \dots, \quad \mathbf{h}^{(K)} = a(\boldsymbol{\beta}_{K-1} + \Omega_{K-1} \mathbf{h}^{(K-1)}),$$

$$\mathbf{y} = \boldsymbol{\beta}_K + \Omega_K \mathbf{h}^{(K)}, \quad \Phi = \{(\boldsymbol{\beta}_k, \Omega_k)\}_{k=0}^K.$$

Hyperparameters:

- K : number of hidden layers
- D_k : number of hidden units in layer k

Training still uses gradient descent to update weights and biases.

2.8 Linear Region Capacity: Shallow vs Deep

For 1D input, a shallow ReLU network with D hidden units has at most $D + 1$ linear regions.

For deep ReLU networks (input dimension d_0), a standard lower-bound style expression is

$$R_{\text{deep}} \gtrsim \left(\prod_{k=1}^{K-1} \left\lfloor \frac{D_k}{d_0} \right\rfloor^{d_0} \right) \sum_{j=0}^{d_0} \binom{D_K}{j}.$$

This captures why depth can create many more linear regions than a single hidden layer.

3 Lecture 3: Likelihood, Loss Functions, and Probabilistic Outputs

3.1 Continuation from Shallow to Deep Models

Recall the shallow mapping

$$\mathbf{x} \mapsto y, \quad y = f(\mathbf{x}; \phi).$$

Deep neural networks are models with more than one hidden layer.

First hidden layer in vector form:

$$\mathbf{h}^{(1)} = a(\beta_0 + \Omega_0 \mathbf{x}).$$

Each hidden unit is a weighted combination of input variables, followed by a nonlinearity.

3.2 Dimension Check for Matrix Multiplication

To ensure multiplication is valid, keep dimensions explicit:

$$\mathbf{x} \in \mathbb{R}^{d_0}, \quad \Omega_0 \in \mathbb{R}^{D_1 \times d_0}, \quad \beta_0 \in \mathbb{R}^{D_1}, \quad \mathbf{h}^{(1)} \in \mathbb{R}^{D_1}.$$

Then

$$\Omega_0 \mathbf{x} \in \mathbb{R}^{D_1},$$

so adding β_0 is dimensionally correct.

For deeper layers:

$$\mathbf{h}^{(k)} = a\left(\beta_{k-1} + \Omega_{k-1} \mathbf{h}^{(k-1)}\right),$$

with

$$\Omega_{k-1} \in \mathbb{R}^{D_k \times D_{k-1}}, \quad \beta_{k-1} \in \mathbb{R}^{D_k}.$$

3.3 Loss Functions and Best Parameters

Training means finding parameters that best explain data. Given

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I,$$

we optimize a loss

$$L(\mathcal{D}, \phi) = \sum_{i=1}^I \ell(y_i, f(\mathbf{x}_i; \phi)).$$

3.4 Maximum Likelihood Principle

In probabilistic modeling, we use conditional probability

$$p(y_i \mid \mathbf{x}_i, \phi).$$

Assuming independent observations:

$$p(y_1, \dots, y_I \mid \mathbf{x}_1, \dots, \mathbf{x}_I, \phi) = \prod_{i=1}^I p(y_i \mid \mathbf{x}_i, \phi).$$

Maximum likelihood estimator:

$$\hat{\phi} = \arg \max_{\phi} \prod_{i=1}^I p(y_i \mid \mathbf{x}_i, \phi).$$

Take log (monotonic transform):

$$\hat{\phi} = \arg \max_{\phi} \sum_{i=1}^I \log p(y_i \mid \mathbf{x}_i, \phi).$$

Equivalent minimization:

$$\hat{\phi} = \arg \min_{\phi} \left[- \sum_{i=1}^I \log p(y_i \mid \mathbf{x}_i, \phi) \right].$$

3.5 From Gaussian Likelihood to Least Squares

For regression, assume

$$y_i \mid \mathbf{x}_i, \phi \sim \mathcal{N}(f(\mathbf{x}_i; \phi), \sigma^2).$$

Then

$$p(y_i \mid \mathbf{x}_i, \phi) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y_i - f(\mathbf{x}_i; \phi))^2}{2\sigma^2}\right).$$

Negative log-likelihood over dataset:

$$- \sum_{i=1}^I \log p(y_i \mid \mathbf{x}_i, \phi) = \frac{I}{2} \log(2\pi\sigma^2) + \frac{1}{2\sigma^2} \sum_{i=1}^I (y_i - f(\mathbf{x}_i; \phi))^2.$$

Since constants do not depend on ϕ :

$$\hat{\phi} = \arg \min_{\phi} \sum_{i=1}^I (y_i - f(\mathbf{x}_i; \phi))^2.$$

So least squares is MLE under Gaussian noise with mean $\mu = f(\mathbf{x}; \phi)$ and standard deviation σ .

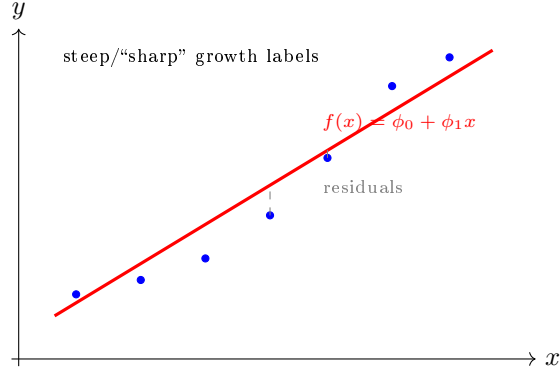


Figure 5: Least-squares line fit to data with increasing slope trend.

3.6 Diagram: Slope-Based Regression (Least Squares)

3.7 Binary Classification and Bernoulli Likelihood

For binary targets $y \in \{0, 1\}$ and Bernoulli parameter λ :

$$p(y | \lambda) = \begin{cases} 1 - \lambda, & y = 0, \\ \lambda, & y = 1. \end{cases}$$

Equivalent compact form:

$$p(y | \lambda) = (1 - \lambda)^{1-y} \lambda^y, \quad \lambda \in [0, 1].$$

Use a network score $f(\mathbf{x}; \phi)$ and map it with sigmoid:

$$\hat{\lambda} = \sigma(f(\mathbf{x}; \phi)), \quad \sigma(z) = \frac{1}{1 + \exp(-z)}.$$

3.8 Diagram: Probability Growth with Sigmoid

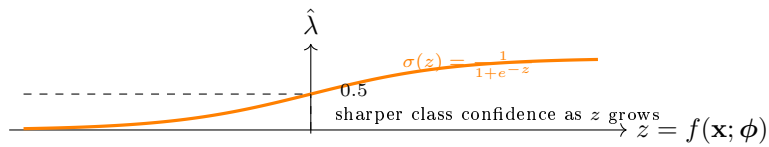


Figure 6: Sigmoid converts score to Bernoulli probability.

3.9 Multiclass Classification and Softmax

For K classes with logits $f_k(\mathbf{x}; \phi)$:

$$p(y = k \mid \mathbf{x}, \phi) = \text{softmax}_k(\mathbf{f}(\mathbf{x}; \phi)) = \frac{\exp(f_k(\mathbf{x}; \phi))}{\sum_{j=1}^K \exp(f_j(\mathbf{x}; \phi))}.$$

3.10 Cross-Entropy as Negative Log-Likelihood

For one-hot target vector $\mathbf{t}_i$ and predicted class probabilities $\hat{\mathbf{p}}_i$:

$$\mathcal{L}_{\text{CE}} = - \sum_{i=1}^I \sum_{k=1}^K t_{ik} \log \hat{p}_{ik}.$$

This is exactly the negative log-likelihood for categorical outputs.

3.11 Theoretical vs Empirical Distribution and KL

Let p^* be the true (theoretical) data distribution and $\hat{p}_{\text{emp}}$ the empirical distribution from samples. Cross-entropy decomposition:

$$H(p^*, \hat{p}_\phi) = H(p^*) + D_{\text{KL}}(p^* \parallel \hat{p}_\phi),$$

where

$$D_{\text{KL}}(p \parallel q) = \sum_x p(x) \log \frac{p(x)}{q(x)}.$$

Since $H(p^*)$ is constant in ϕ , minimizing cross-entropy minimizes KL divergence to the target distribution.

3.12 Dirac Delta View of Empirical Distribution

For continuous data $\{\mathbf{x}_i\}_{i=1}^I$:

$$\hat{p}_{\text{emp}}(\mathbf{x}) = \frac{1}{I} \sum_{i=1}^I \delta(\mathbf{x} - \mathbf{x}_i),$$

where $\delta(\cdot)$ is the Dirac delta function. This formalizes empirical distribution as probability mass concentrated on observed samples.

4 Lecture 4: Fitting Models and Optimization Algorithms

4.1 Fitting Models

We want to fit a model family to data by optimizing a loss function. Dataset:

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^I.$$

Model:

$$\hat{y}_i = f(\mathbf{x}_i; \phi).$$

Loss:

$$L(\phi) = \sum_{i=1}^I \ell_i(\phi) = \sum_{i=1}^I \ell(y_i, f(\mathbf{x}_i; \phi)).$$

Goal:

$$\hat{\phi} = \arg \min_{\phi} L(\phi).$$

4.2 Probabilistic View and Maximum Likelihood

Step 1: model conditional probability

$$p(y \mid \mathbf{x}, \boldsymbol{\theta}).$$

Step 2: parameterize with a network output

$$\boldsymbol{\theta}_i = f(\mathbf{x}_i; \boldsymbol{\psi}).$$

Step 3: maximize log-likelihood:

$$\hat{\boldsymbol{\psi}} = \arg \max_{\boldsymbol{\psi}} \sum_{i=1}^I \log p(y_i \mid \boldsymbol{\theta}_i).$$

Equivalent minimization:

$$\hat{\boldsymbol{\psi}} = \arg \min_{\boldsymbol{\psi}} \left[- \sum_{i=1}^I \log p(y_i \mid \boldsymbol{\theta}_i) \right].$$

Gaussian example:

$$y_i \mid \mathbf{x}_i \sim \mathcal{N}(\mu_i, \sigma^2), \quad \mu_i = f(\mathbf{x}_i; \phi).$$

Then minimizing negative log-likelihood gives least squares.

4.3 Gradient Descent (GD)

Update rule:

$$\phi_{t+1} = \phi_t - \alpha \nabla_{\phi} L(\phi_t),$$

where α is the step size (learning rate).

Three-step view:

- Step 0: initialize ϕ_0
- Step 1: compute gradient $\nabla_{\phi} L(\phi_t)$
- Step 2: update $\phi \leftarrow \phi - \alpha \nabla_{\phi} L$

4.4 Square Loss Example (Linear Regression)

Model:

$$f(x; \phi) = \phi_0 + \phi_1 x.$$

Loss:

$$L(\phi) = \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i)^2.$$

Full derivatives:

$$\begin{aligned} \frac{\partial L}{\partial \phi_0} &= 2 \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i), \\ \frac{\partial L}{\partial \phi_1} &= 2 \sum_{i=1}^I (\phi_0 + \phi_1 x_i - y_i) x_i. \end{aligned}$$

Vector update:

$$\begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix}_{t+1} = \begin{bmatrix} \phi_0 \\ \phi_1 \end{bmatrix}_t - \alpha \begin{bmatrix} \partial L / \partial \phi_0 \\ \partial L / \partial \phi_1 \end{bmatrix}_t.$$

4.5 Convex vs Non-Convex Loss

Convex losses have one global minimum (for linear least squares in standard form). Deep models are generally non-convex and can have many local minima/saddle points.

4.6 Stochastic Gradient Descent (SGD)

Instead of full dataset each step, use a mini-batch $B_t \subset \{1, \dots, I\}$:

$$\mathbf{g}_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla_{\phi} \ell_i(\phi_t), \quad \phi_{t+1} = \phi_t - \alpha \mathbf{g}_t.$$

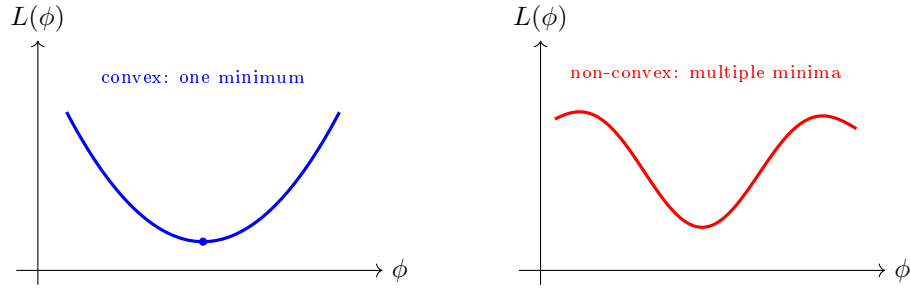


Figure 7: Optimization landscape: convex (left) vs non-convex (right).

Epoch: one full pass over all training examples. If dataset size is I and batch size is b , then one epoch has about

$$\left\lceil \frac{I}{b} \right\rceil$$

parameter updates.

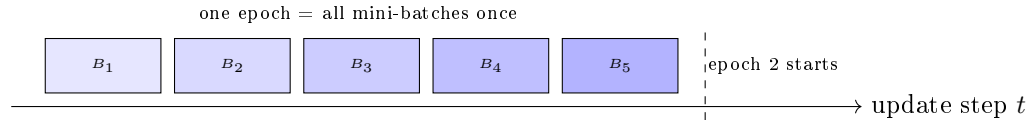


Figure 8: Mini-batch SGD timeline and the meaning of an epoch.

4.7 Hyperparameters

Hyperparameters are chosen outside optimization, for example:

- learning rate α
- batch size b
- number of epochs
- optimizer parameters $(\beta, \beta_1, \beta_2, \epsilon)$

4.8 SGD with Momentum

Momentum tracks an exponentially weighted average of gradients:

$$\mathbf{m}_{t+1} = \beta \mathbf{m}_t + (1 - \beta) \mathbf{g}_t, \quad \mathbf{g}_t = \frac{1}{|B_t|} \sum_{i \in B_t} \nabla \ell_i(\phi_t),$$

$$\phi_{t+1} = \phi_t - \alpha \mathbf{m}_{t+1}.$$

Interpretation: updates become smoother and continue in consistent descent directions, reducing zig-zag behavior. Momentum includes history and can be viewed as an infinite weighted sum of past gradients.

Example up to m_3 . Assume scalar gradients and $\beta = 0.9$, $m_0 = 0$, with $g_1 = 2.0$, $g_2 = 1.0$, $g_3 = 1.5$:

$$m_1 = 0.9 \cdot 0 + 0.1 \cdot 2.0 = 0.2,$$

$$m_2 = 0.9 \cdot 0.2 + 0.1 \cdot 1.0 = 0.28,$$

$$m_3 = 0.9 \cdot 0.28 + 0.1 \cdot 1.5 = 0.402.$$

So the update at step 3 uses αm_3 , not just αg_3 .

4.9 Graph: Without vs With Momentum

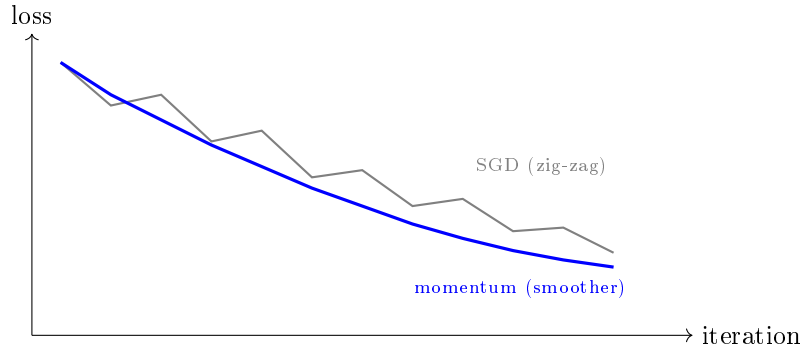


Figure 9: Momentum smooths SGD updates by using past gradient information.

4.10 Nesterov Accelerated Gradient (NAG)

Nesterov evaluates gradient at a look-ahead point:

$$\tilde{\phi}_t = \phi_t - \alpha \beta \mathbf{m}_t,$$

$$\mathbf{m}_{t+1} = \beta \mathbf{m}_t + (1 - \beta) \nabla L(\tilde{\phi}_t),$$

$$\phi_{t+1} = \phi_t - \alpha \mathbf{m}_{t+1}.$$

Compared with classical momentum, NAG reacts earlier to changes in curvature.

4.11 Adam (Adaptive Moment Estimation)

Adam combines momentum (first moment) and adaptive scaling (second moment):

$$\begin{aligned}\mathbf{m}_{t+1} &= \beta_1 \mathbf{m}_t + (1 - \beta_1) \mathbf{g}_t, \\ \mathbf{v}_{t+1} &= \beta_2 \mathbf{v}_t + (1 - \beta_2) \mathbf{g}_t \odot \mathbf{g}_t.\end{aligned}$$

Bias-corrected moments:

$$\hat{\mathbf{m}}_{t+1} = \frac{\mathbf{m}_{t+1}}{1 - \beta_1^{t+1}}, \quad \hat{\mathbf{v}}_{t+1} = \frac{\mathbf{v}_{t+1}}{1 - \beta_2^{t+1}}.$$

Update:

$$\phi_{t+1} = \phi_t - \alpha \frac{\hat{\mathbf{m}}_{t+1}}{\sqrt{\hat{\mathbf{v}}_{t+1}} + \epsilon}.$$

Here ϵ stabilizes division, and β_1, β_2 control averaging. This gives per-parameter step-size control using squared gradients.

4.12 Practical Notes

- GD is stable but expensive on large datasets.
- SGD is fast per step but noisy.
- Momentum and Nesterov improve directionality and speed.
- Adam is often robust with minimal tuning, especially in deep networks.
- Depth is fixed by architecture; optimizer hyperparameters control training behavior.